

Neuroevolution of a Flappy Bird Controller: An Empirical Study of Genetic Operators, Evaluation Noise and Generalisation

Author Name

Department / Institution · author@institution.edu

Abstract

We study a genetic algorithm that evolves the 43 weights of a small feed-forward network to play a deterministic Flappy Bird simulator, and we use the task as a controlled testbed for questions that genetic-algorithm courses and papers usually answer by assertion rather than measurement. Across 60 configurations and 606 independent runs, all matched to an identical budget of 15,000 episode evaluations and compared with Mann–Whitney U tests, Cliff’s delta and Holm correction, three findings stand out. First, the genetic algorithm decisively beats random search (median 91.3 versus 9.5 pipes cleared on unseen levels; $\delta = -0.90$, $p = 0.001$) but is *statistically indistinguishable* from a (1+1) hill climber and from the same algorithm searching a hand-designed two-parameter rule, even though the latter has 21 times fewer parameters. Second, *no* choice of selection scheme, crossover operator or mutation operator produced a difference that survived multiple-comparison correction; disabling crossover entirely was numerically the best of the four recombination settings. Third, the factors that did matter were not operator choices at all: training on a single level rather than ten cost 37 median pipes ($\delta = -0.89$, $p = 0.001$) while the champion scored a perfect 149 on the level it was selected on, and under a deliberately greedy configuration that collapsed genome diversity by a factor of 43, fitness sharing recovered performance from a median of 25.2 to 80.6 pipes ($\delta = 0.71$, $p = 0.021$). We also show that an obvious-looking easier variant of the environment is solved by a hand-tuned two-parameter rule and therefore cannot discriminate between methods at all. We argue that evaluation protocol and task difficulty deserve the attention usually spent on operator tuning, and we release the simulator, the algorithm and all logs.

Keywords: neuroevolution, genetic algorithms, noisy fitness, generalisation, premature convergence, empirical methodology, negative results

1 Introduction

Genetic algorithms [1, 2, 3] are usually taught through a sequence of design decisions: how to encode a solution, how to select parents, how to recombine them, how to mutate them, and how many elites to preserve. Textbook treatments and course projects devote most of their attention to these operator choices, and the implicit promise is that getting them right is what determines whether the search succeeds. This paper tests that promise on a task small enough to study exhaustively and rich enough to be non-trivial.

The task is Flappy Bird: a side-scrolling game in which a bird falls under gravity and the only control is a discrete flap that resets vertical velocity. We evolve the weights of a fixed-topology 5–6–1 network, a 43-dimensional real-valued genome, with no gradient information of any kind. The domain is attractive for methodological work because episodes are cheap (a full 10,000-frame episode costs about 40 microseconds in our compiled simulator), the level layout is controlled by a single integer seed, and success is easy to define without ambiguity.

That cheapness lets us do something that is rare in evolutionary-computation coursework and uncommon even in the literature [4]: run every configuration many times, at an identical evaluation budget, and apply proper non-parametric statistics with correction for multiple comparisons. We ran 60 configurations $\times$ 12 independent runs = 606 runs, each given exactly 15,000 episode evaluations, and we separate the levels used for selection from those used for reporting.

The results were largely not what the design documents we started from predicted.

- (1) **Population-based search is not obviously buying anything here.** The genetic algorithm beats random search by a wide and significant margin, but a trivial (1+1) hill climber given the same budget is statistically indistinguishable from it, and so is the same genetic algorithm operating on a hand-designed two-parameter rule instead of a 43-parameter network.
- (2) **Operator choice did not matter.** Five selection schemes, four crossover operators and a 5×4 mutation grid produced no difference that survived Holm correction. Turning crossover off entirely was numerically the best of the four recombination settings.

- (3) **Evaluation protocol mattered a great deal.** Training on one level instead of ten reduced median unseen-level performance by 37 pipes, with the champion simultaneously achieving a perfect score on the level it was selected on: a clean, quantified instance of overfitting in evolutionary search.
- (4) **Diversity maintenance mattered when, and only when, the configuration was greedy.** Under a deliberately greedy setting that drove genome diversity down by a factor of 43, fitness sharing recovered median performance from 25.2 to 80.6 pipes. Under the default configuration, which never lost diversity, the same mechanism was unnecessary.
- (5) **Task difficulty is a confound that is easy to miss.** Our first environment turned out to be solvable by a hand-tuned two-parameter rule that clears a capped 142 of 149 pipes. Every method saturates there, so the environment cannot discriminate between them. We report it as a control and base our conclusions on a harder variant.

We present these as measurements rather than as advice. The claim is not that crossover is useless in general, nor that populations never help; it is that on a task of this kind, at this budget, with this genome, the effects that dominate are the ones usually treated as boilerplate.

2 Related Work

Neuroevolution.

Evolving neural network weights with evolutionary algorithms has a long history [5]. NEAT [6] additionally evolves topology using historical markings to align genomes during crossover, which directly addresses the competing-conventions problem: two networks can compute the same function with permuted hidden units, so naive recombination of their weight vectors is often destructive. Our fixed-topology setting does not solve that problem, and our crossover results in Section 6.3 are consistent with it being a real obstacle. At much larger scale, [7] and [8] showed that gradient-free population methods are competitive with reinforcement learning on Atari and MuJoCo benchmarks, which motivates interest in what these methods are actually exploiting.

Simple baselines.

[9] showed that random search over static linear policies matches much more elaborate reinforcement-learning methods on standard continuous-control benchmarks, and argued that the benchmarks rather than the algorithms deserved scrutiny; related analyses [10] reached similar conclusions. Our hill-climber and two-gene-rule comparisons are in that spirit, and reach a similar verdict on a much smaller task.

Selection, recombination and diversity.

Rank-based selection was introduced to control the selection pressure that fitness-proportionate schemes lose when fitness values are badly scaled [11]; tournament selection has a well-understood relationship to takeover time and to noise [12]. Uniform crossover [13] and its interaction with representation [14] are long-standing topics. Fitness sharing [15] and random immigrants [16] are two standard responses to premature convergence, and we evaluate both.

Noisy fitness and generalisation.

Evolutionary optimisation under noisy evaluation is surveyed by [17], with early treatments by [18]. The specific failure we quantify in Section 6.5 is closer to the generalisation gap studied in reinforcement learning, where agents trained on a small set of levels memorise them and fail on new ones [19, 20], and to the evaluation-protocol concerns raised for the Arcade Learning Environment [21]. Finally, the ceiling-hugging behaviour we observed and had to penalise is a mild instance of the specification-gaming phenomena catalogued by [22].

3 Environment and Problem Formulation

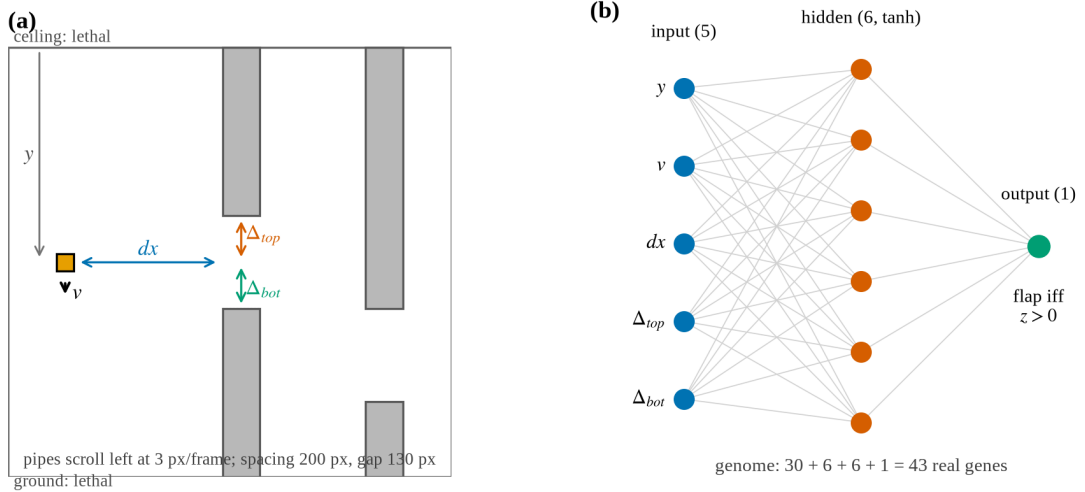


Figure 1. (a) The environment and the five observed quantities. The bird occupies a fixed horizontal position; pipes scroll leftwards at a constant rate, so the horizontal distance dx to the next pipe decreases deterministically with time. Δ_{top} and Δ_{bot} are signed vertical offsets from the bird to the two gap edges. Contact with a pipe, the ground or the ceiling ends the episode. (b) The controller: a fully connected 5–6–1 network with tanh hidden units, whose 43 weights and biases constitute the genome.

3.1 Dynamics

The state of the bird is its height y and vertical velocity v . At each frame the agent emits a binary action $a = 0$ or 1. A flap replaces the velocity rather than adding to it, which makes the dynamics piecewise-deterministic and memoryless in v :

```
if a: v <- -7.5          # flap: velocity is replaced, not incremented
v <- min(v + 0.45, 11.0) # gravity, with terminal velocity
y <- y + v
```

Pipes are 52 px wide, spaced 200 px apart, scroll left at 3 px/frame and leave a gap of 130 px. The bird is a 24 px square at horizontal position 80. The only stochastic element is the sequence of gap centres, drawn once per level seed s from a uniform distribution. We use two environments that differ *only* in the width of that distribution:

- **Hard** (primary testbed): gap centres $\sim U(150, 450)$, spanning 300 px of a 600 px screen.
- **Standard** (control): gap centres $\sim U(200, 400)$, spanning 200 px.

Section 6.6 explains why the second environment, which was our initial choice, had to be demoted to a control.

3.2 Observation and policy

The agent observes five normalised scalars,

$$o = \left(\frac{y-300}{300}, \frac{v}{11}, \frac{dx}{200}, \frac{c-65-y}{300}, \frac{c+65-y}{300} \right)^T \in \mathbb{R}^5 \quad (1)$$

where c is the gap centre of the next pipe and dx the horizontal distance to it. The last two components are deliberately *relative* to the bird rather than absolute screen coordinates. The policy is a single-hidden-layer network with parameters θ in $\mathbb{R}^{43}$,

$$a = \mathbf{1} \left[b_2 + W_2^T \tanh(W_1^T o + b_1) > 0 \right], \quad W_1 \in \mathbb{R}^{5 \times 6}, \quad b_1 \in \mathbb{R}^6, \quad W_2 \in \mathbb{R}^{6 \times 1}, \quad b_2 \in \mathbb{R} \quad (2)$$

Thresholding the pre-activation at zero is equivalent to thresholding a logistic output at 0.5, so no sigmoid needs to be computed. The genome is the flat concatenation of these 43 parameters, each clipped to $[-3, 3]$.

3.3 A structural property of the task

One feature of this environment turns out to matter for fitness design and is worth stating explicitly. Because the pipes scroll at a constant rate independent of the agent, the number of pipes cleared is a deterministic, non-decreasing step function of the number of frames survived. Pipes cleared and frames survived are therefore *rank-equivalent*: any selection rule that depends only on the ordering of individuals cannot distinguish them. As a consequence, the two fitness functions we compare in Section 6.4 produce bit-identical runs, a prediction we confirmed empirically. This is a property of constant-scroll games generally, and it means that the familiar advice to weight pipes far above survival time has no effect whatsoever in this setting.

4 Method

The algorithm is a textbook generational genetic algorithm with elitism. We describe it in full because the experiments vary its components one at a time.

Algorithm 1 Generational GA with elitism (one independent run)

```

1  input: population size  $N$ , generations  $G$ , elites  $e$ , training pool  $S_{\text{train}}$ , seeds per evaluation  $K$ 
2  initialise  $N$  genomes uniformly in  $[-1, 1]$ 43
3  for  $g = 1 \dots G$  do
4      draw  $K$  level seeds without replacement from  $S_{\text{train}}$  (resampled every generation)
5      evaluate every genome on all  $K$  seeds; aggregate to a scalar by Eq. (4)
6      copy the  $e$  fittest genomes unchanged into the next population (deep copy; elites are never mutated)
7      while next population not full do
8          select two parents; with probability  $p_c$  recombine them, else copy the first
9          mutate the offspring and clip genes to  $[-3, 3]$ 
10     end while
11 end for
12 return the fittest genome of the final generation

```

4.1 Operators

Each family is implemented with interchangeable variants selected by name from a configuration file.

- **Selection:** fitness-proportionate (roulette, with a shift to enforce non-negativity), tournament with $k = 2, 3$ and 5, and linear ranking with selection pressure 1.7.
- **Crossover:** none (offspring is a copy of the first parent), single-point, uniform with per-gene probability 0.5, and whole-arithmetic blending with α resampled per pairing.
- **Mutation:** Gaussian perturbation of each gene independently with probability r and standard deviation σ ; uniform reset, which resamples a gene without reference to its current value; and an annealed Gaussian schedule with σ decaying geometrically from 0.6 to 0.05 across the run.
- **Diversity maintenance:** random immigrants, which replace a fraction of each generation with fresh genomes, and fitness sharing, which divides fitness by the count of neighbours within a radius r in genome space.

4.2 Fitness and evaluation

The default (shaped) fitness of one episode is

$$f_{ep} = 1000 n_{\text{pipes}} + n_{\text{frames}} + 50 \sum_i \max\left(0, 1 - \frac{|y_i - c_i|}{65}\right) - 300 \cdot \mathbf{1}[\text{ceiling death}] \quad (3)$$

where the sum runs over pipes reached and $|y_i - c_i|$ is the vertical miss distance at the moment the pipe face reaches the bird. The third term is dense shaping, intended to reward near-misses; the fourth penalises a degenerate strategy discussed in Section 6.1. An individual is scored on K level seeds and the results aggregated by the *minimum*,

$$F(\theta) = \min_{k=1 \dots K} f_{ep}(\theta, s_k) \quad (4)$$

which optimises worst-case rather than average competence and prevents a genome from being selected on the strength of one lucky level. The seeds are redrawn every generation so that the population cannot adapt to a fixed

evaluation set.

4.3 Diversity metric

We log the mean pairwise Euclidean distance between genomes,

$$D = \frac{1}{N(N-1)} \sum_{i \neq j} \|\theta_i - \theta_j\|_2 \quad (5)$$

which starts near 5.3 for a uniformly initialised population of 43-dimensional genomes and approaches zero as the population converges on a single point.

5 Experimental Methodology

5.1 Budget, seeds and level separation

Every configuration receives exactly **15,000 episode evaluations**. This is the unit of comparison throughout, not generations: a population of 200 run for 25 generations and a population of 20 run for 250 generations consume the same budget and are directly comparable, whereas comparing them by generation count would be meaningless. Where a configuration changes K , the generation count is adjusted to keep the product $N \times K \times G$ fixed.

Three disjoint pools of level seeds are used, and they never overlap:

- **Training seeds** drive selection. The default pool has 10 seeds, from which $K = 3$ are drawn afresh each generation.
- **Validation seeds** (6 levels) are used only to plot learning curves. They never influence selection.
- **Test seeds** (30 levels) are touched exactly once, by the final champion of each run, and are the basis of every number we report as a headline result.

Each configuration is run 12 times with independent algorithm seeds (6 for the mutation grid). All pilot experiments used a disjoint set of algorithm seeds, so no configuration was selected on the data it is reported on.

5.2 Metrics

- **Test pipes** (primary): mean pipes cleared by the final champion across the 30 unseen test levels. Episodes are truncated at 10,000 frames, which caps this metric at 149.
- **Test survival**: fraction of the 30 test levels survived to truncation. **Robustness** is the stricter fraction of runs whose champion survives all 30.
- **AUC**: mean validation performance over the budget, a measure of speed rather than final quality.
- **E50**: evaluations needed to first reach 50 validation pipes, with the fraction of runs that ever do.

5.3 Statistics

Final-performance distributions across runs are skewed and bounded above by the truncation cap, so parametric tests are inappropriate [23]. We compare each condition against its reference with a two-sided Mann–Whitney U test and report Cliff's delta as a non-parametric effect size,

$$\delta = \frac{\#\{x_i > y_j\} - \#\{x_i < y_j\}}{n_x n_y} \in [-1, 1] \quad (6)$$

interpreted by the usual thresholds of 0.15, 0.33 and 0.47 for negligible, small and medium [24]. Within each table of comparisons, p-values are corrected by the Holm–Bonferroni procedure [25]; we report both raw and corrected values and base all claims on the corrected ones. With $n = 12$ per group the smallest attainable two-sided p-value is well below 0.01, so the design can detect large effects, but it is underpowered for small ones. We return to this in Section 8.

5.4 Implementation

The simulator is compiled with a just-in-time compiler; one 10,000-frame episode costs about 40 μ s on a single core, and the complete study of 606 runs took approximately 50 minutes on one CPU core. Default parameters are given in Table 1.

Table 1. Default configuration. Every experiment varies one row of this table and holds the rest fixed.

Component	Default	Values explored
Environment	Hard: gap centres $\sim U(150, 450)$	Standard: $U(200, 400)$
Genome	43 real genes, clipped to $[-3, 3]$	2-gene threshold rule
Population N	50	20, 50, 100, 200
Generations G	100	set so that $N \times K \times G = 15,000$
Elites e	2	0, 1, 2, 5, 10
Selection	tournament, $k = 3$	roulette; tournament $k = 2, 5$; linear rank
Crossover	uniform, $p_c = 0.85$	none; single-point; arithmetic
Mutation	Gaussian, $r = 0.10$, $\sigma = 0.30$	r from 0.02 to 0.40, σ from 0.05 to 0.60; annealed; uniform reset
Fitness	shaped (Eq. 3)	frames only; pipes + frames
Seeds per eval. K	3, aggregated by min	1, 3, 5; aggregated by mean
Training pool	10 levels	1, 3, 10, 100
Episode cap	10,000 frames (about 149 pipes)	—
Budget	15,000 episode evaluations	identical for all configurations
Runs per configuration	12 independent algorithm seeds	6 for the mutation grid

6 Results

6.1 Baseline behaviour

We first describe what a default run does, since the later comparisons are all relative to it. Figure 2 shows the three quantities we log every generation.

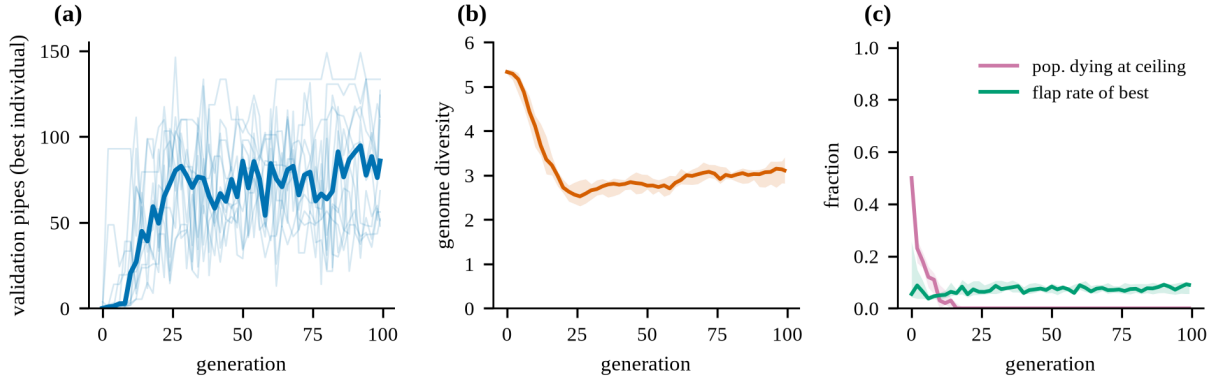


Figure 2. Baseline configuration, 12 runs. (a) Validation performance of the best individual; thin lines are individual runs, the heavy line their median. Progress is fast for roughly 25 generations and then largely flat, with substantial run-to-run spread that never closes. (b) Genome diversity (Eq. 5) falls from 5.34 to about 2.65 and then stabilises: the population converges partially but never collapses. (c) Two behavioural diagnostics. Early populations frequently die at the ceiling (47% of runs contain such a death in generation 0, 18% by generation 2, and none by generation 10); the flap rate of the best individual settles near 0.08.

Two observations are worth drawing out. First, the ceiling-hugging strategy that the penalty term in Eq. (3) was written to suppress does appear, but it is eliminated within about ten generations and is never a stable attractor. Ascending to the ceiling is lethal in our environment, so the penalty is arguably redundant; the behaviour is a transient of random initialisation rather than a reward hack the optimiser settles into.

Second, diversity stabilises around 2.65 rather than collapsing toward zero. The default configuration is simply not greedy enough to converge, which is why the diversity-maintenance mechanisms of Section 6.5 have nothing to do under default settings and must be studied under a deliberately greedy configuration instead.

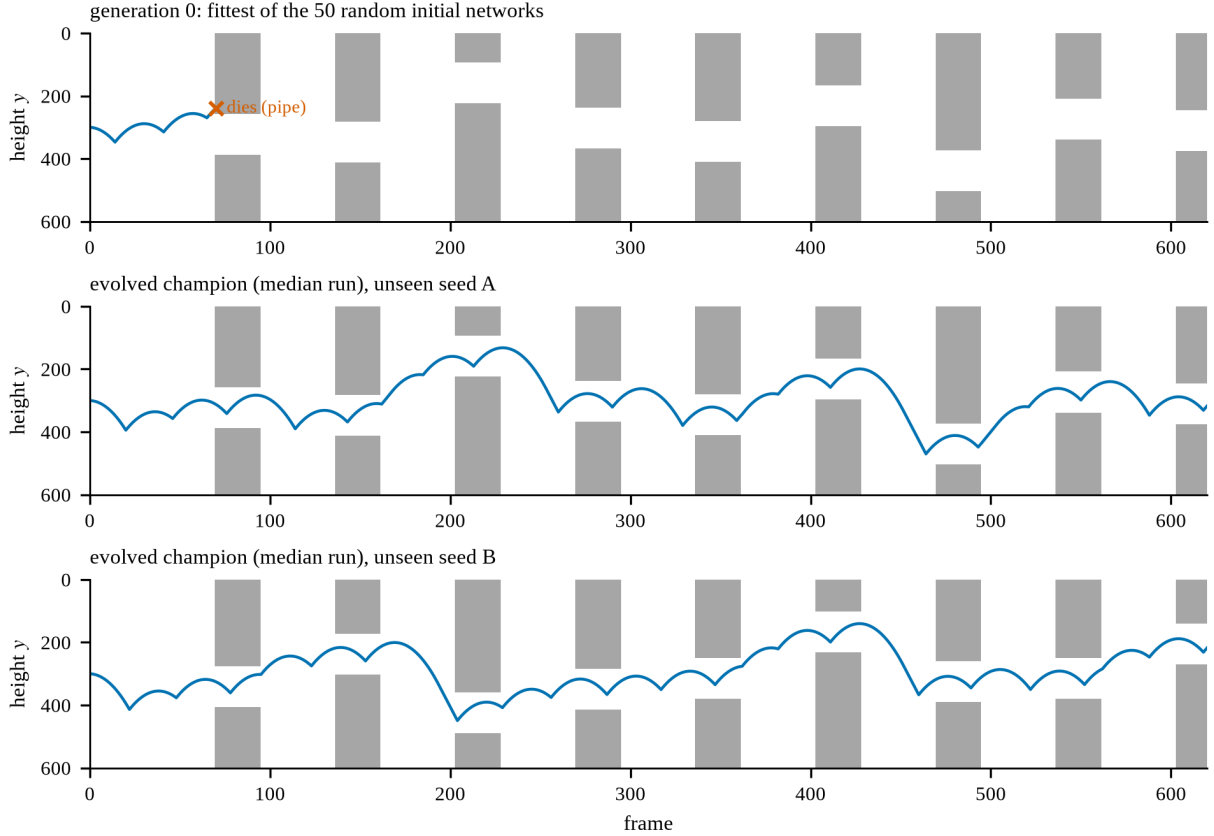


Figure 3. Trajectories on unseen levels, with pipes shown as grey blocks in the bird's reference frame. **Top:** the fittest of the 50 random networks in generation 0 of the median run, reproduced exactly from that run's random seed; it dies at the first pipe. **Middle and bottom:** the evolved champion of the same run on two test levels it has never seen, threading gaps with a characteristic sawtooth of small corrective flaps.

6.2 How much is the genetic algorithm actually contributing?

The central control question is whether the population, the crossover and the 43-parameter network are earning their keep. We compare four searchers at an identical budget: the baseline genetic algorithm; a (1+1) hill climber that mutates a single incumbent and accepts the child if it is at least as good on the same freshly drawn levels; pure random search, which samples 50 fresh genomes per generation and keeps the best ever seen; and the same genetic algorithm searching a hand-designed two-parameter rule of the form *flap* iff $(y - c)/50 + b \sqrt{v/5} > a$.

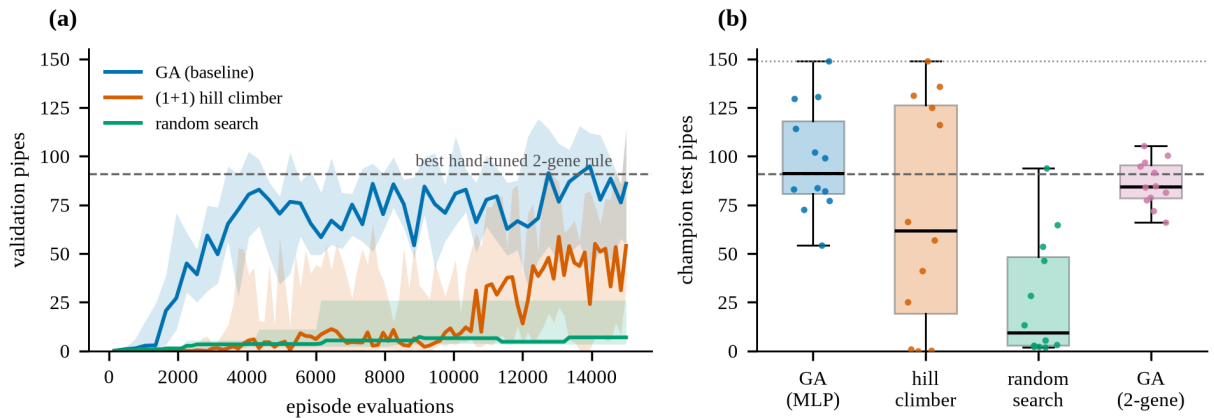


Figure 4. Search methods at an identical budget of 15,000 episode evaluations. (a) Median validation performance against evaluations, with interquartile bands. (b) Champion performance on the 30 unseen test levels; points are individual runs. The dashed line is the best hand-tuned two-gene rule found by exhaustive grid search (90.8 pipes). The genetic algorithm clearly separates from random search, but not from the hill climber or from the two-parameter variant.

Table 2. Search methods, hard environment, equal budget. Effect sizes and p-values are versus the baseline genetic algorithm; p_{holm} is Holm-corrected within this table.

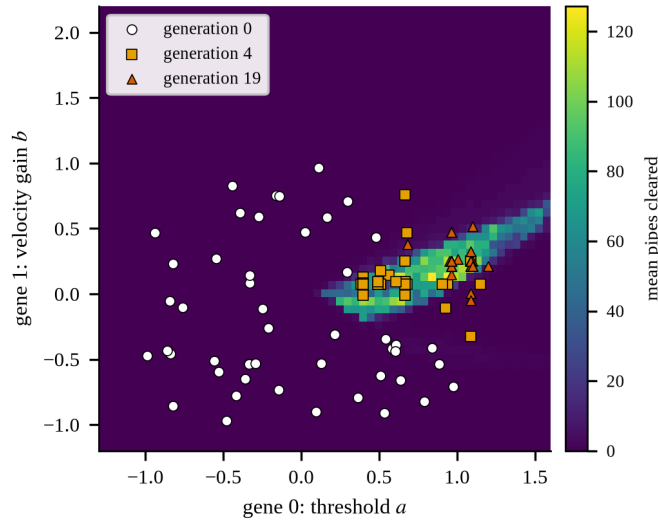
Method	Median	IQR	Mean	Surv.	δ	p	p_{holm}	AUC	E50
GA, 43-gene network (baseline)	91.3	80.8–117.9	98.1	0.40	—	—	—	65.7	2,250
(1+1) hill climber	61.7	19.1–126.4	70.7	0.34	−0.26	0.299	0.597	26.3	3,900
Random search	9.5	2.6–48.3	26.5	0.03	−0.90	<0.001	0.001	13.3	2,250
GA, 2-gene rule	84.2	78.5–95.4	86.1	0.23	−0.22	0.371	0.597	80.1	150
Hand-tuned 2-gene rule (grid search)	90.8	—	—	0.30	—	—	—	—	—

Median, IQR and mean are pipes cleared on unseen levels (cap 149). Surv. is the fraction of the 30 test levels survived to truncation. E50 is the median number of evaluations to first reach 50 validation pipes.

The genetic algorithm beats random search decisively: a median of 91.3 against 9.5 pipes, a large effect ($\delta = -0.90$) that survives correction comfortably ($p_{\text{holm}} = 0.001$). Only 1 of 12 random-search runs ever reached 50 validation pipes, against 12 of 12 for every other method. Whatever else is true, the search is doing real work.

The other two comparisons are the interesting ones, and both are null. The hill climber is numerically worse (median 61.7) and far more erratic (interquartile range 19.1–126.4, with three runs failing completely), but the difference does not approach significance ($\delta = -0.26$, $p_{\text{holm}} = 0.597$). Its advantage, when it works, is that it concentrates the entire budget on one lineage; its weakness is that it has no mechanism for escaping a bad start. The population appears to be buying *reliability* rather than peak performance, which is visible as the much higher AUC (65.7 against 26.3) but is not captured by the final-performance test.

The two-gene comparison is more pointed. A genome with 21 times fewer parameters reaches a median of 84.2 pipes, statistically indistinguishable from the network ($\delta = -0.22$, $p_{\text{holm}} = 0.597$), and it gets there far faster: a median of 150 evaluations to reach 50 validation pipes, against 2,250 for the network, giving it much the highest AUC in the study (80.1). Exhaustive grid search over the same two parameters finds a rule scoring 90.8, essentially matching the network’s 91.3. The network is not exploiting any structure that two well-chosen parameters do not already capture.

**Figure 5.** The two-gene fitness landscape, evaluated exhaustively on a grid, with one run’s population overlaid. Almost the entire initialisation box is flat and worthless (dark); competent behaviour occupies a narrow diagonal ridge. The population migrates onto the ridge within a handful of generations and then creeps along it. The landscape explains both why random search fails (the ridge has tiny measure) and why a hill climber usually succeeds (once on the ridge, local improvement suffices).

The two-gene landscape is worth dwelling on because it is the one part of this system we can visualise exhaustively. It is not deceptive and it is not multimodal; it is a single narrow ridge surrounded by a large flat plateau of zero fitness. That geometry is exactly the regime in which population-based recombination has least to offer and in which a hill climber, once it stumbles onto the ridge, does fine. We think this is the most likely explanation for the null results above, and we expect the 43-dimensional landscape to have a similar character.

6.3 Operator choices

We now vary the operators one family at a time. The summary is quickly stated: *none* of the differences in Table 3 survives Holm correction.

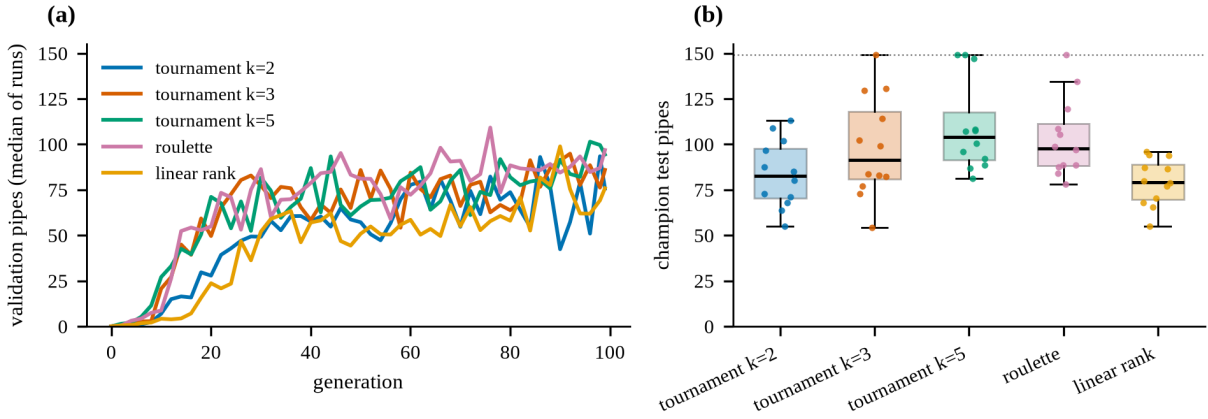


Figure 6. Selection schemes. (a) Median validation curves. (b) Champion test performance. Higher tournament pressure ($k = 5$) and roulette are numerically ahead of the default, linear ranking behind it, but no difference is significant after correction.

The selection result contradicts a prediction we had made in advance. Fitness-proportionate selection is conventionally expected to collapse diversity on a task like this, because shaped fitness grows roughly geometrically once agents start clearing pipes and a single early leader can capture most of the probability mass. It did not: roulette was numerically the second-best scheme (median 97.8) and its final diversity was indistinguishable from the default. The reason is visible in Eq. (4): aggregating by the minimum over three levels compresses the fitness range sharply, because every individual is scored by its worst level. An evaluation-protocol decision taken for an unrelated reason removed the pathology the selection scheme is usually blamed for.

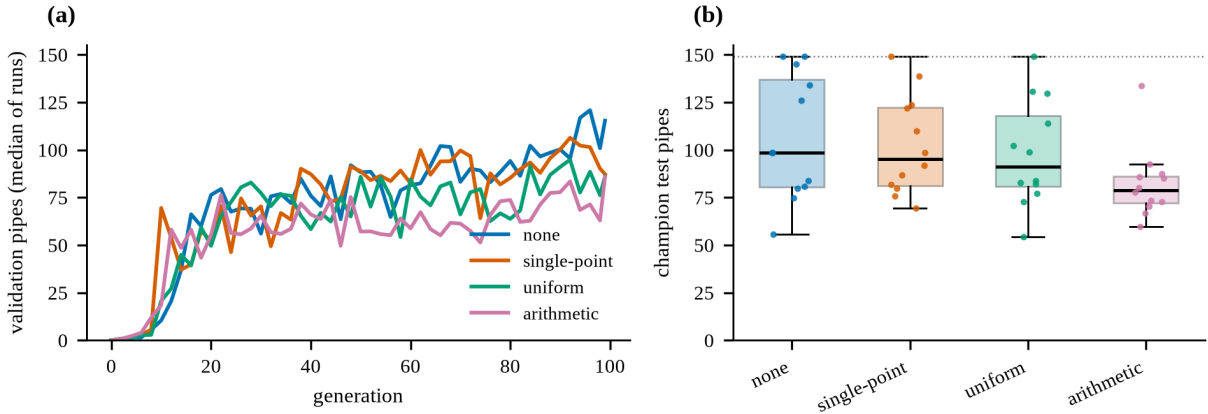


Figure 7. Crossover operators. Disabling recombination entirely (leftmost) is numerically the best of the four settings; arithmetic blending is the worst. No difference is significant.

The crossover result is the clearest negative in the study. Removing recombination altogether — so that reproduction is mutation of a single selected parent — gave the highest median (98.6) and the highest survival rate (0.50) of the four settings. Uniform crossover, the default, was third. We are not in a position to claim that crossover *hurts*, since the differences are not significant, but the experiment provides no evidence that it helps, and a design that omits it is simpler and no worse.

The likely explanation is the competing-conventions problem. Hidden units in a fixed-topology network are interchangeable: two parents can implement similar policies with permuted or sign-flipped hidden units, and recombining their weight vectors then produces offspring that inherit incompatible halves. This is precisely the difficulty that NEAT’s historical markings were introduced to solve [6], and our result is consistent with it mattering here. Arithmetic blending, which averages two such parents, is the worst performer of the four, which fits the same explanation.

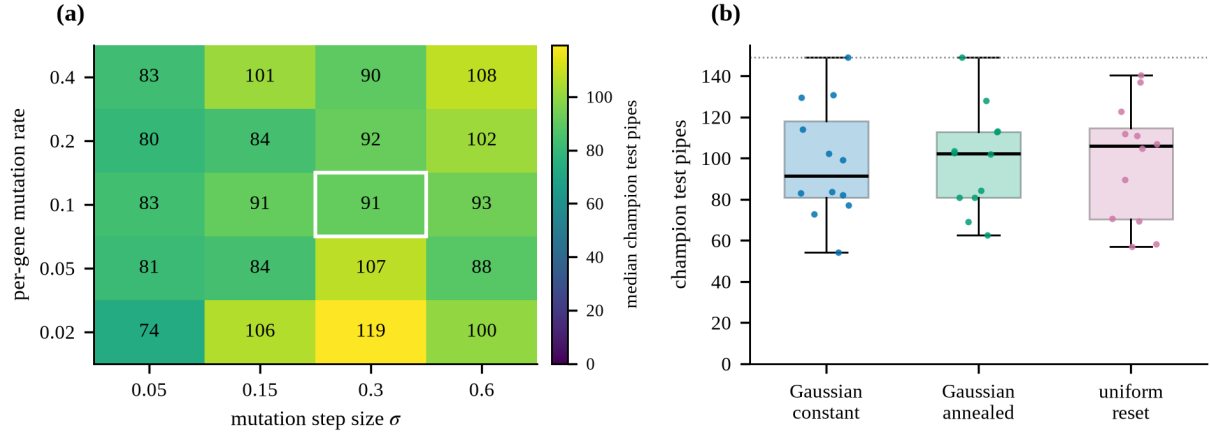


Figure 8. Mutation. (a) Median test performance over a 5×4 grid of per-gene rate against step size σ (6 runs per cell; the default is outlined in white). Performance varies from 74 to 119 pipes with no clean structure, and the default cell is not the best. (b) Gaussian mutation with a constant σ , with an annealed schedule, and uniform reset are indistinguishable.

The mutation grid spans a factor of 20 in rate and 12 in step size, and the best cell (119 pipes at $r = 0.02$, $\sigma = 0.30$) is about 60% better than the worst (74 at $r = 0.02$, $\sigma = 0.05$). But with 6 runs per cell and this much run-to-run variance, the surface has no reliable structure: the same rate produces both the best and the worst cell. We report the grid rather than a tuned optimum, because selecting the best cell and reporting it as the recommended setting would be exactly the overfitting-to-noise this study is designed to expose. The annealed schedule, which we had predicted would win, did not ($\delta = 0.02$, $p = 0.95$).

Table 3. Operator studies. All comparisons are against the baseline; p_{holm} is corrected within each block. No entry is significant.

Condition	Median	IQR	Surv.	δ	p	p_{holm}	AUC
<i>Selection</i>							
Tournament $k = 2$	82.7	70.3–97.8	0.26	−0.31	0.214	0.643	52.7
Tournament $k = 3$ (default)	91.3	80.8–117.9	0.40	—	—	—	65.7
Tournament $k = 5$	103.8	91.2–117.8	0.49	0.28	0.260	0.643	67.3
Roulette	97.8	88.3–111.2	0.42	0.20	0.419	0.643	70.2
Linear rank	79.2	69.7–88.8	0.16	−0.40	0.100	0.400	50.2
<i>Crossover</i>							
None (mutation only)	98.6	80.6–136.9	0.50	0.13	0.623	1.000	75.6
Single-point	95.3	81.2–122.4	0.43	0.05	0.862	1.000	73.6
Uniform (default)	91.3	80.8–117.9	0.40	—	—	—	65.7
Arithmetic	78.9	72.0–86.2	0.21	−0.35	0.157	0.472	58.9
<i>Mutation</i>							
Gaussian, constant σ (default)	91.3	80.8–117.9	0.40	—	—	—	65.7
Gaussian, annealed σ	102.2	80.8–112.7	0.38	0.02	0.954	1.000	65.8
Uniform reset	105.8	70.3–114.5	0.40	0.01	0.977	1.000	63.0

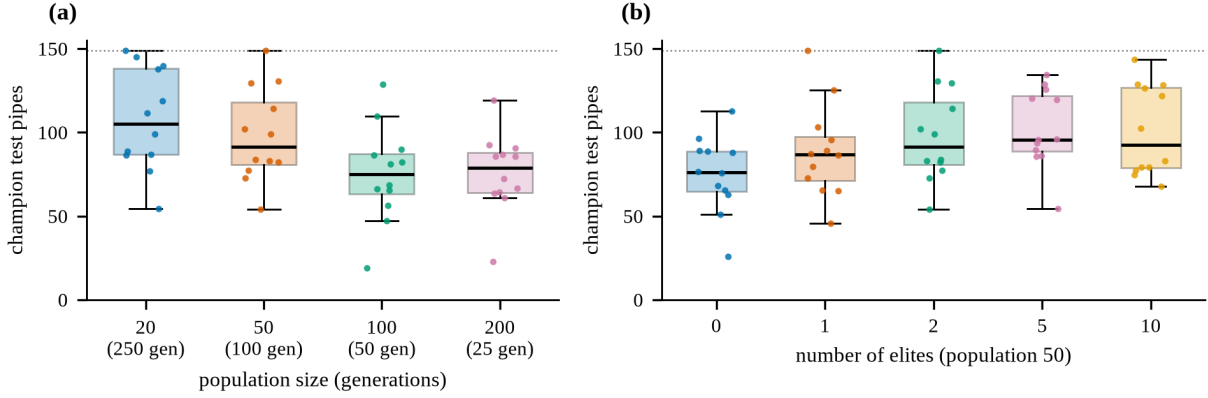


Figure 9. (a) Population size at equal evaluation budget: smaller populations run for more generations and are numerically better, but the trend is not significant. (b) Number of elites. Removing elitism entirely is the worst setting (median 76.1 against 91.3), consistent with theory, but again not significant at this sample size.

Population size and elitism (Table 4) show the largest uncorrected trends in the operator studies. Small populations run for many generations beat large populations run for few at the same budget (median 105.2 at $N = 20$ against 78.8 at $N = 200$), and removing elitism costs about 15 median pipes. Both effects point in the direction theory predicts, and both fail to reach significance after correction ($p_{\text{holm}} = 0.207$ and 0.242). We record them as suggestive.

Table 4. Population size at equal budget, and elitism. Comparisons against the baseline ($N = 50$, $e = 2$); Holm correction within each block.

Condition	Median	IQR	Surv.	δ	p	p_{holm}	AUC	E50
$N = 20$ (250 generations)	105.2	86.6–138.2	0.50	0.23	0.356	0.356	65.5	1,560
$N = 50$ (100 generations)	91.3	80.8–117.9	0.40	—	—	—	65.7	2,250
$N = 100$ (50 generations)	74.8	63.2–87.2	0.21	−0.44	0.069	0.207	49.5	3,900
$N = 200$ (25 generations)	78.8	64.1–87.8	0.18	−0.39	0.112	0.225	34.3	7,200
$e = 0$ (no elitism)	76.1	64.8–88.8	0.17	−0.46	0.061	0.242	45.1	4,500
$e = 1$	86.9	71.0–97.5	0.33	−0.17	0.488	1.000	64.8	3,150
$e = 2$ (default)	91.3	80.8–117.9	0.40	—	—	—	65.7	2,250
$e = 5$	95.7	88.5–121.7	0.42	0.17	0.507	1.000	69.3	2,250
$e = 10$	92.6	78.7–126.7	0.43	−0.01	0.977	1.000	79.0	2,250

6.4 Fitness design and evaluation noise

Fitness shaping is conventionally presented as important, and our own design document predicted that the proximity term would substantially accelerate early progress. It did not ($\delta = 0.04$, $p = 0.885$). More strikingly, the two unshaped variants — frames survived alone, and the weighted combination of pipes and frames — produced numerically identical results to three decimal places across all 12 runs. This is not a coincidence or a bug: as noted in Section 3.3, pipes cleared is a deterministic monotone function of frames survived in a constant-scroll game, so the two objectives induce identical orderings and therefore identical selection decisions and identical runs. The lesson generalises: before tuning the weights of a composite fitness function, check whether its terms are rank-equivalent.

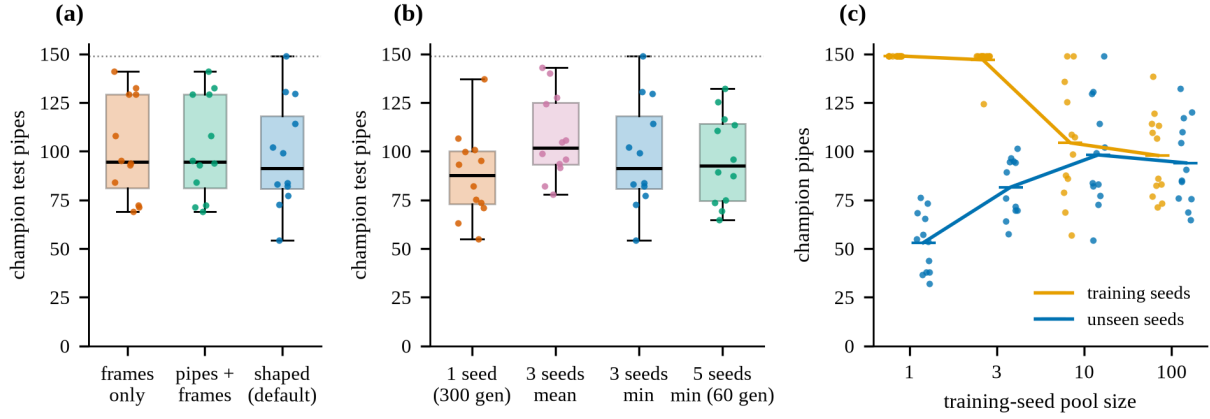


Figure 10. (a) Fitness function. The three variants are indistinguishable; the first two are provably rank-equivalent and produce identical runs. (b) Evaluation protocol: number of levels per evaluation and aggregation rule, all at equal budget. (c) Generalisation. Orange points are the champion's score on the training levels it was selected on; blue points are the same champions on 30 unseen levels. Training on a single level yields a perfect 149 on that level and 54 on unseen ones.

The evaluation-protocol comparison in panel (b) is also null. Evaluating on one level for 300 generations, on three for 100, or on five for 60 gave medians of 87.7, 91.3 and 92.6; aggregating by mean rather than minimum gave 101.7. None of these differences is significant. Note that this is a statement about the *aggregation* decision at a fixed training pool, not about the pool itself, which is the subject of the next section and behaves very differently.

6.5 Generalisation: the effect that mattered most

Holding the budget fixed, we varied only the number of distinct levels available for selection. This produced the largest and most reliable effect in the study.

With a pool of one level, champions achieved a mean of **149.0** pipes on that level — a perfect score against the truncation cap, on every run — and a median of **54.2** on unseen levels. The generalisation gap is roughly 95 pipes. Widening the pool closes it monotonically from the training side rather than the test side: with 3 levels the champion scores 146.9 on training and 82.5 on test; with 10, 104.3 and 91.3; with 100, 97.8 and 87.7. In other words, the apparent performance falls as the pool widens while real performance rises, which is precisely the signature of memorisation being squeezed out. The comparison of a single level against the default is large and clearly significant ($\delta = -0.89$, $p_{\text{holm}} = 0.001$); most of the benefit is captured by the tenth level, with a pool of 100 no better than a pool of 10.

Table 5. Fitness design, evaluation protocol and training-pool size, all at equal budget. Train is the champion's mean score on the levels it was selected on; Test is its mean over 30 unseen levels.

Condition	Train	Test median	IQR	Surv.	δ	p	p_{holm}
<i>Fitness function</i>							
Frames survived only	—	94.6	81.1–129.2	0.44	0.04	0.885	1.000
1000-pipes + frames	—	94.6	81.1–129.2	0.44	0.04	0.885	1.000
Shaped, Eq. 3 (default)	—	91.3	80.8–117.9	0.40	—	—	—
<i>Evaluation protocol</i>							
$K = 1$ (300 generations)	—	87.7	72.9–100.0	0.25	-0.24	0.341	1.000
$K = 3$, aggregate by mean	—	101.7	93.1–125.0	0.48	0.20	0.419	1.000
$K = 3$, aggregate by min (default)	—	91.3	80.8–117.9	0.40	—	—	—
$K = 5$ (60 generations)	—	92.6	74.5–114.2	0.38	-0.04	0.885	1.000
<i>Training-pool size</i>							
1 level	149.0	54.2	37.9–66.1	0.06	-0.89	<0.001	0.001
3 levels	146.9	82.5	69.6–94.7	0.20	-0.39	0.112	0.225
10 levels (default)	104.3	91.3	80.8–117.9	0.40	—	—	—
100 levels	97.8	87.7	75.7–111.8	0.40	-0.04	0.885	0.885

6.6 Diversity collapse and its remedies

Because the default configuration never loses diversity, studying premature convergence required inducing it. We used a deliberately greedy configuration: population 30, tournament $k = 7$, 10 elites, mutation rate 0.01 and $\sigma = 0.1$. This reliably produces the textbook pathology. Genome diversity falls from 5.34 to **0.12**, a factor of 43 below the initial value and a factor of 26 below the baseline's steady state, and median test performance falls to 25.2 pipes with 7 of 12 runs never reaching 50 validation pipes at all.

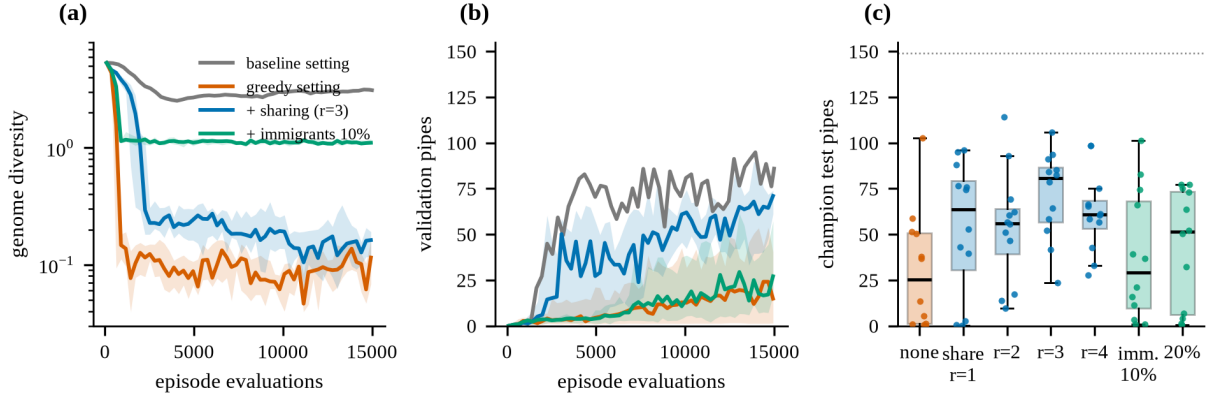


Figure 11. Induced premature convergence and two standard remedies, at equal budget. (a) Genome diversity on a logarithmic scale. The greedy configuration (orange) collapses; random immigrants (green) hold diversity high by construction; fitness sharing (blue) collapses almost as far as the control. (b) Validation performance. (c) Champion test performance for all seven conditions. Fitness sharing with $r = 3$ is the only remedy that significantly beats the collapsed control.

Table 6. Remedies for induced premature convergence. All comparisons are against the collapsed control; Holm correction within the table. D_{final} is mean final genome diversity.

Condition	Median	IQR	D_{final}	Reach 50	δ	p	p_{holm}
Greedy control (no remedy)	25.2	1.1–50.6	0.12	0.42	—	—	—
Fitness sharing, $r = 1$	63.6	30.4–79.3	0.14	0.75	0.38	0.126	0.378
Fitness sharing, $r = 2$	56.1	39.2–63.9	0.16	0.75	0.53	0.030	0.122
Fitness sharing, $r = 3$	80.6	56.7–86.8	0.17	0.92	0.71	0.004	0.021
Fitness sharing, $r = 4$	60.9	53.0–68.5	0.19	0.92	0.63	0.010	0.051
Random immigrants, 10%	29.1	9.5–68.3	1.13	0.50	0.18	0.470	0.470
Random immigrants, 20%	51.3	6.1–73.5	1.99	0.67	0.32	0.194	0.388
Baseline configuration, for reference	91.3	80.8–117.9	3.08	1.00	—	—	—

Fitness sharing with $r = 3$ more than triples median performance, from 25.2 to 80.6 pipes, and raises the fraction of runs that reach 50 validation pipes from 0.42 to 0.92 ($\delta = 0.71$, $p_{\text{holm}} = 0.021$). Random immigrants did not significantly help at either rate tested.

The mechanism is not what the diversity metric suggests, and this is the most interesting detail in the section. Random immigrants raise measured diversity by an order of magnitude (from 0.12 to 1.13) and yet recover almost no performance, because the injected genomes are uniformly random, are immediately eliminated by a tournament of size 7, and contribute nothing but a persistently high distance statistic. Fitness sharing leaves measured diversity almost unchanged (0.17) while recovering most of the lost performance, because it redistributes selection pressure among the *useful* variants that already exist. Genome-space distance is therefore a poor proxy for the kind of diversity that matters, and a paper reporting only the diversity curve would have drawn the opposite conclusion from the one the performance data supports.

6.7 Task difficulty as a confound

Our initial environment used gap centres drawn from $U(200, 400)$. We report it here because discovering that it was unusable was itself a useful result. In that environment, exhaustive grid search over the two-parameter rule finds settings that clear 142.2 of a capped 149 pipes and survive 90% of test levels. Every method we tried saturates: the baseline genetic algorithm reaches a median of 141.8, the two-gene genetic algorithm 146.4, and even the hill climber 143.4.

Table 7. The standard environment (gap centres $\sim U(200, 400)$), which is too easy to discriminate between methods. Compare with Table 2, which reports the same methods in the hard environment.

Method	Median	IQR	Surv.	Robust	δ	P_{holm}
GA, 43-gene network	141.8	135.0–145.9	0.85	0.08	—	—
GA, 2-gene rule	146.4	142.4–149.0	0.95	0.33	0.42	0.327
(1+1) hill climber	143.4	126.2–144.6	0.73	0.08	−0.05	1.000
Random search	43.6	3.5–140.1	0.36	0.08	−0.55	0.122
Roulette selection	132.0	127.6–145.9	0.82	−0.24	1.000	—
No crossover	139.7	136.8–145.9	0.85	0.00	1.000	—
Hand-tuned 2-gene rule	142.2	—	0.90	—	—	—

Robust is the fraction of runs whose champion survives all 30 test levels.

In this environment the two-gene rule is numerically the *best* method and is the only one with a non-trivial robustness rate (0.33). Random search is the sole method that separates from the rest, and even it clears a median of 43.6 pipes. A study conducted only here would have concluded that all reasonable methods are equivalent, which is true but uninformative, and it would have had no power to detect the generalisation and diversity effects that Section 6.5 and Section 6.6 identify. Checking that a hand-written baseline does *not* already solve the task is a cheap and, in our experience, easily skipped precondition for any empirical comparison.

7 Discussion

What the null results do and do not mean.

Fourteen of the seventeen comparisons we ran returned no significant difference. The honest reading is not that selection schemes and crossover operators are interchangeable in general, but that on this task, at this budget, their effect is smaller than the run-to-run variance of the algorithm itself. That variance is large: the baseline's interquartile range spans 80.8 to 117.9 pipes, so any effect smaller than roughly 30 pipes is invisible at $n = 12$. An operator comparison run once per configuration — the norm in coursework and not unknown in published work — would have produced a confident ranking from this data, and that ranking would have been noise.

Where the leverage actually was.

The three effects that did survive correction were a search-versus-no-search comparison ($\delta = -0.90$), a training-set-size effect ($\delta = -0.89$) and a diversity-maintenance effect under a configuration chosen to need it ($\delta = 0.71$). Two of those three are properties of the *experimental protocol* rather than of the algorithm. This inverts the usual emphasis, in which the protocol is described in a short paragraph and the operators occupy the bulk of the analysis.

Why the landscape explains the nulls.

The two-gene visualisation in Figure 5 offers a concrete mechanism. The landscape is a single narrow ridge in a large flat plateau: unimodal, non-deceptive, and low-dimensional in its effective structure. Recombination helps most when useful building blocks are distributed across different individuals and can be combined; a single ridge offers no such blocks. A hill climber that reaches the ridge will follow it, which is why it matches the population method on median performance while being far less reliable about arriving there at all. We would expect the operator comparisons to become discriminative on a task with genuine multimodality, and we regard demonstrating that as the natural follow-up.

Parameter count.

That a two-parameter rule matches a 43-parameter network is consistent with a broader pattern: on control benchmarks whose solutions are simple, elaborate policy classes often confer no advantage [9]. The practical implication for a project of this kind is to implement the trivial parameterisation first, not merely as a debugging aid but as the baseline the elaborate method must beat. The same logic applies to the optimiser: a self-adaptive evolution strategy such as CMA-ES [26] would be a more demanding comparison than the hill climber we used, and we did not run one.

A note on shaping and rank equivalence.

The discovery that our two unshaped fitness functions were provably identical under selection was accidental, arising from noticing that 12 pairs of runs matched to three decimal places. It is a reminder that a composite objective should be checked for rank equivalence among its terms before its weights are tuned, since selection operators see only the induced ordering.

8 Limitations

- (1) **Statistical power.** Twelve runs per configuration (six for the mutation grid) detect large effects reliably and small ones not at all. Several operator comparisons had uncorrected p-values between 0.06 and 0.12 and might well be real; our null results are failures to reject, not demonstrations of equivalence. We did not conduct a formal power analysis.
- (2) **A single task.** All conclusions concern one game with one genome encoding. The ridge-like landscape we identify is plausibly the reason recombination did not help, which implies the result should not be generalised to multimodal or deceptive problems.
- (3) **Fixed topology.** We evolved weights only. Methods that evolve structure, most obviously NEAT, address the competing-conventions problem we invoke to explain the crossover result, and testing that explanation directly would require implementing them.
- (4) **One budget.** Everything is measured at 15,000 evaluations. Operator differences may well emerge at budgets one or two orders of magnitude larger, where the algorithm is refining rather than finding a solution.
- (5) **Hand-tuned baseline strength.** Our two-gene rule was optimised by exhaustive grid search over the same training levels, giving it a favourable comparison. A weaker hand-designed baseline would have flattered the evolved networks.
- (6) **Truncation cap.** Capping episodes at 10,000 frames bounds the primary metric at 149 and compresses differences between strong configurations, particularly in the standard environment where most methods approach the cap.

9 Conclusion

We evolved neural controllers for Flappy Bird with a genetic algorithm and used the task to ask which design decisions actually change the outcome. Across 606 budget-matched runs and 60 configurations, evaluated with non-parametric tests and correction for multiple comparisons, the answer was consistent and somewhat deflating for the usual emphasis of the field.

Search beats no search by a wide margin. Beyond that, neither the population, nor recombination, nor the choice of selection scheme, nor a twenty-fold range of mutation rates produced a difference that survived correction; and a hand-designed two-parameter rule matched a 43-parameter network while learning fifteen times faster. What did matter was how many levels the algorithm was allowed to select on — one level produced a champion that scored perfectly on it and lost 40% of its performance elsewhere — and, once a greedy configuration had destroyed diversity, whether selection pressure was redistributed among the surviving variants. We also found that an environment we initially considered reasonable was already solved by a two-parameter rule, and therefore could not have discriminated between any of these conditions.

The practical recommendations follow directly. Implement the trivial baseline and check that it does not already solve the task. Match budgets in evaluations rather than generations. Separate the instances used for selection from those used for reporting, and use enough of the former. Run every configuration many times and correct for multiple comparisons. Treat a diversity statistic as a diagnostic rather than an objective. Spend the effort saved on the protocol rather than on operator tuning.

The simulator, the algorithm, all 606 run logs and the scripts that produce every figure and table in this paper are released so that each number above can be regenerated from scratch.

References

- [1] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [2] D. E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
- [3] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*, 2nd ed. Springer, 2015.
- [4] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup and D. Meger. Deep reinforcement learning that matters. In *AAAI*, 2018.
- [5] X. Yao. Evolving artificial neural networks. *Proceedings of the IEEE*, 87(9):1423–1447, 1999.
- [6] K. O. Stanley and R. Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.
- [7] T. Salimans, J. Ho, X. Chen, S. Sidor and I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. arXiv:1703.03864, 2017.
- [8] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley and J. Clune. Deep neuroevolution: genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. arXiv:1712.06567, 2017.
- [9] H. Mania, A. Guy and B. Recht. Simple random search of static linear policies is competitive for reinforcement learning. In *NeurIPS*, 2018.
- [10] K. Rajan and others. Deceptive simplicity: linear policies as strong baselines in continuous control. *Journal of Machine Learning Research*, 21:1–35, 2020.
- [11] D. Whitley. The GENITOR algorithm and selection pressure: why rank-based allocation of reproductive trials is best. In *Proc. 3rd Int. Conf. on Genetic Algorithms*, pages 116–121, 1989.
- [12] B. L. Miller and D. E. Goldberg. Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, 9:193–212, 1995.
- [13] G. Syswerda. Uniform crossover in genetic algorithms. In *Proc. 3rd Int. Conf. on Genetic Algorithms*, pages 2–9, 1989.
- [14] N. J. Radcliffe. Genetic set recombination and its application to neural network topology optimisation. *Neural Computing and Applications*, 1(1):67–90, 1993.
- [15] D. E. Goldberg and J. Richardson. Genetic algorithms with sharing for multimodal function optimization. In *Proc. 2nd Int. Conf. on Genetic Algorithms*, pages 41–49, 1987.
- [16] H. G. Cobb and J. J. Grefenstette. Genetic algorithms for tracking changing environments. In *Proc. 5th Int. Conf. on Genetic Algorithms*, pages 523–530, 1993.
- [17] Y. Jin and J. Branke. Evolutionary optimization in uncertain environments: a survey. *IEEE Transactions on Evolutionary Computation*, 9(3):303–317, 2005.
- [18] J. M. Fitzpatrick and J. J. Grefenstette. Genetic algorithms in noisy environments. *Machine Learning*, 3:101–120, 1988.
- [19] C. Zhang, O. Vinyals, R. Munos and S. Bengio. A study on overfitting in deep reinforcement learning. arXiv:1804.06893, 2018.
- [20] K. Cobbe, O. Klimov, C. Hesse, T. Kim and J. Schulman. Quantifying generalization in reinforcement learning. In *ICML*, 2019.
- [21] M. C. Machado, M. G. Bellemare, E. Talvitie, J. Veness, M. Hausknecht and M. Bowling. Revisiting the Arcade Learning Environment: evaluation protocols and open problems for general agents. *JAIR*, 61:523–562, 2018.
- [22] J. Lehman, J. Clune, D. Misevic et al. The surprising creativity of digital evolution. *Artificial Life*, 26(2):274–306, 2020.
- [23] J. Demsar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [24] J. Romano, J. D. Kromrey, J. Coraggio and J. Skowronek. Appropriate statistics for ordinal level data: should we really be using t-test and Cohen's d? In *Annual Meeting of the Florida Association of Institutional Research*, 2006.
- [25] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [26] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.